

1. [앱 → 서버] HTTP API (인증 및 설정)

앱에서 회원가입, 로그인, 로봇/구역 등록 등을 처리하는 기본 API입니다.

1-1. 회원가입 (POST /api/auth/signup)

- 기능: 새로운 유저 계정을 생성합니다.
- [앱 발신] Body (JSON):
- JSON

```
{
  "user_id": "user123",
  "password": "password123!",
  "name": "홍길동",
  "phone_number": "010-1234-5678",
  "country": "KR",
  "email": "user@example.com"
}
```

- [서버 수신] 성공 응답: 201 Created / {"status": "success", "message": "회원가입이 완료되었습니다."}

1-2. 로그인 (POST /api/auth/login)

- 기능: 유저 로그인 및 정보 확인.
- [앱 발신] Body (JSON):
- JSON

```
{
  "user_id": "user123",
  "password": "password123!"
}
```

- [서버 수신] 성공 응답:
- JSON

```
{
  "status": "success",
  "message": "홍길동님 환영합니다!",
  "data": { "user_id": "user123", "name": "홍길동" }
}
```

1-3. 신규 로봇 등록 (POST /api/robot/register)

- 기능: 유저의 계정에 새로운 로봇(하드웨어)을 귀속시킵니다.
- [앱 발신] Body (JSON):

- JSON

```
{
  "user_id": "user123",
  "robot_id": "R_001"
}
```

1-4. 구역 등록 및 수정 (POST /api/zone/setup)

- 기능: 농장의 특정 구역을 새로 만들거나, 기존 구역의 정보를 수정합니다. (기존 `marker_list` 파라미터는 삭제되었습니다.)

- [앱 발신] Body (JSON):

- JSON

```
{
  "user_id": "user123",
  "zone_id": "zone01",
  "zone_name": "A동 1열",
  "min_lat": 37.1234,
  "max_lat": 37.1235,
  "min_lng": 126.1234,
  "max_lng": 126.1235
}
```

- [서버 수신] 성공 응답: `{"status": "success", "message": "'A동 1열' 구역이 성공적으로 등록(또는 수정)되었습니다."}`

1-5. 구역 일괄 삭제 (POST /api/zone/delete)

- 기능: 구역을 완전히 삭제합니다. 주의: 이 구역에 속해 있던 마커(Marker)들도 DB에서 한꺼번에 자동 삭제됩니다.

- [앱 발신] Body (JSON):

- JSON

```
{
  "user_id": "user123",
  "zone_id": "zone01"
}
```

- [서버 수신] 성공 응답: `{"status": "success", "message": "'A동 1열' 구역과 연관된 모든 마커가 삭제되었습니다."}`

1-6. 마커 등록 및 수정 (POST /api/marker/setup)

- 기능: 로봇이 실내에서 인식할 마커를 생성하고, 특정 구역(Zone)에 소속시킵니다. 이미 있는 마커라면 구역을 이동시키거나 좌표를 수정합니다.
- [앱 발신] Body (JSON):
- JSON

```
{
  "marker_id": "M001",
  "zone_id": "zone01",
  "lat": 37.12345, // (선택 사항) 실외 GPS 마커일 경우 위도
  "lng": 126.12345 // (선택 사항) 실외 GPS 마커일 경우 경도
}
```

-
-
- [서버 수신] 성공 응답: `{"status": "success", "message": "마커(M001)가 성공적으로 등록(또는 수정)되었습니다."}`
- 에러 응답 예시 (404): 만약 zone01이 아직 만들어지지 않은 구역이라면, "존재하지 않는 구역(Zone)입니다. 먼저 구역을 생성하세요."라는 에러를 반환하여 데이터 꼬임을 방지합니다.

1-7. 개별 마커 삭제 (POST /api/marker/delete)

- 기능: 특정 마커 1개만 콕 집어서 삭제합니다. (구역은 그대로 남습니다.)
- [앱 발신] Body (JSON):
- JSON

```
{
  "marker_id": "M001"
}
```

-
-
- [서버 수신] 성공 응답: `{"status": "success", "message": "마커(M001)가 삭제되었습니다."}`

1-5. 재배 데이터 완전 삭제 (POST /api/zones/batch/manage)

- 기능: 특정 구역(또는 전체)의 누적된 작물 재배 데이터를 DB에서 초기화합니다.
- [앱 발신] Body (JSON):
- JSON

```
{
  "user_id": "user123",
  "zone_id": "zone01" // 전체 초기화 시 "ALL"
}
```

2. [앱 → 서버] HTTP API (대시보드 및 통계)

앱의 메인 화면(대시보드)을 그리기 위해 서버에서 데이터를 가져오는(GET) API입니다.
Body 없이 URL 파라미터를 사용합니다.

2-1. 특정 로봇의 현재 상태 조회 (GET /api/robot/status/{robot_id})

- 기능: 로봇의 배터리, 작동 상태, 현재 위치를 가져옵니다. (30초 이상 통신 두절 시 자동 'OFFLINE' 처리)
- URL 예시: /api/robot/status/R_001?user_id=user123
- [서버 응답] 데이터 (JSON):
- JSON

```
{
  "status": "success",
  "data": {
    "robot_id": "R_001",
    "operating_status": "ACTIVE",
    "battery": 85,
    "last_marker_id": "M2",
    "lat": 37.12345,
    "lng": 126.12345,
    "last_updated": "2026-08-07 10:20:00"
  }
}
```

-
-

2-2. 유저 전체 구역 및 재배 이력 조회 (GET /api/user/zones)

- 기능: 내 농장의 모든 구역 정보와, 그 안에 심어진 작물 리스트를 한 번에 가져옵니다.
- URL 예시: /api/user/zones?user_id=user123
- [서버 응답] 데이터 (JSON):
- JSON

```
{
```

```

"status": "success",
"data": [
  {
    "zone_id": "zone01",
    "zone_name": "A동 1열",
    "batches": [
      {
        "batch_id": "zone01_202608...",
        "crop_id": "설향딸기",
        "growth_status": "GROWING",
        "health_status": "NORMAL"
      }
    ]
  }
]
}

```

2-3. 작물 상태 그룹 자동 집계(통계) (GET /api/user/crop/summary)

- 기능: 특정 작물(예: 설향딸기)이 총 몇 개인지, 다 익은 건 몇 개인지 숫자를 집계합니다.
- URL 예시: /api/user/crop/summary?user_id=user123&crop_id=설향딸기
- [서버 응답] 데이터 (JSON):
- JSON

```

{
  "status": "success",
  "data": {
    "crop_id": "설향딸기",
    "total_count": 100,
    "growth_status_counts": { "SEEDLING": 20, "GROWING": 80 },
    "health_status_counts": { "NORMAL": 98, "DISEASED": 2 }
  }
}

```

2-4. 최근 로그 조회 (환경/작물) (GET)

- 환경 로그: /api/logs/env?user_id=user123&limit=10 → 최근 온도/습도 기록 10개 반환, limit 없을시 기본적으로 20개 반환
- 작물 로그: /api/logs/crop?user_id=user123&limit=30 → 최근 이상/촬영 기록 30개 반환, limit 없을시 기본적으로 20개 반환

2-5. 로봇 제어 명령 로그 조회 (GET /api/logs/command)

- 기능: 앱에서 로봇에게 내렸던 최근 제어 명령 이력을 가져옵니다. 원하는 개수를 지정할 수 있습니다.

URL 예시:

- 최근 **10**개만 볼 때: `/api/logs/command?user_id=user123&limit=10`
- 최근 **50**개 볼 때: `/api/logs/command?user_id=user123&limit=50`
- **limit**을 빼고 보내면 알아서 기본 **20**개를 보내줍니다.
- [서버 응답] 데이터 (JSON):
- JSON

```
{
  "status": "success",
  "message": "최근 명령 로그 10개 조회 완료",
  "data": [
    {
      "log_id": 42,
      "robot_id": "R_001",
      "command": "start_patrol",
      "target_zone": "zone02",
      "timestamp": "2026-08-07 13:10:25"
    }
  ]
}
```

3. [로봇 ↔ 서버] MQTT 통신 (하드웨어 제어 및 센서)

로봇(아두이노 등)과 서버가 실시간으로 주고받는 MQTT 메시지 명세입니다.

3-1. [로봇 → 서버] 로봇 실시간 상태 보고 (**handle_robot_status**)

- 기능: 로봇이 주기적으로 서버로 상태를 보고합니다.
- 로봇 송신 토픽: **ddalgi/robot/status**
- [로봇 발신] JSON Payload:
- JSON

```
{
  "robot_id": "R_001",
  "current_zone": "zone01",
  "marker_id": "M2",
  "battery": 82,
  "operating_status": "PATROL",
  "lat": 37.1234,
  "lng": 126.1235
}
```

3-2. [로봇 → 서버] 작물 촬영 로그 보고 (**handle_crop_log**)

- 기능: 로봇이 작물을 인식했을 때, 해당 정보를 서버(ZoneBatch 및 Log)에 누적합니다.
- [로봇 발신] **JSON Payload:**
- JSON

```
{
  "user_id": "user123",
  "robot_id": "R_001",
  "zone_id": "zone01",
  "crop_id": "설향딸기",
  "growth_status": "RIPE",
  "health_status": "NORMAL"
}
```

3-3. [로봇 → 서버] 환경 센서 데이터 보고 (**handle_env_log**)

- 기능: 온습도 센서 측정값을 서버에 기록합니다.
- [로봇 발신] **JSON Payload:**
- JSON

```
{
  "user_id": "user123",
  "robot_id": "R_001",
  "temperature": 24.5,
  "humidity": 65.0
}
```

3-4. [앱 → 서버 → 로봇] 로봇 행동 제어 명령 (**POST /api/robot/command**)

- 기능: 앱에서 버튼을 누르면 서버를 거쳐 로봇에게 직접 명령(순찰, 귀환 등)이 하달됩니다.
- 로봇 수신(구독) 토픽: **ddalgi/robot/command/{robot_id}** (예: **ddalgi/robot/command/R_001**)
- [로봇 수신] **JSON Payload:**
- JSON

```
{
  "command": "start_patrol",
  "target_zone": "zone02",
  "timestamp": "2026-08-07 10:20:00"
}
```

4. HTTP 로봇->서버 작물 사진 업로드 및 데이터 기록 (로봇용)

- 기능: 로봇이 촬영한 작물 사진을 서버(AWS S3)에 올리고, DB(CropLog, ZoneBatch)에 누적 기록합니다. 만약 질병(disease)이 발견되면 즉시 앱으로 MQTT 경고 알림을 발송합니다.
- Method / URL: POST /api/upload/crop
- 전송 방식: multipart/form-data 형식으로 전송해야 합니다.
- [로봇 발신] Form Data 항목:
 - image: (File) 촬영한 이미지 파일 객체 (필수)
 - user_id: (String) 유저 ID (필수)
 - robot_id: (String) 로봇 ID (필수)
 - crop_id: (String) 심어진 작물 (예: "설향딸기")
 - growth_status: (String) 자란 상태 (예: "RIPE", "GROWING")
 - health_status: (String) 건강 상태 (예: "NORMAL", "disease")
 - zone_id: (String) 촬영 구역 (예: "zone01")
- [서버 응답] 데이터 (JSON):
- JSON

```
{
  "status": "success",
  "message": "이미지 업로드 및 로깅 완료",
  "image_url": "https://[버킷명].s3.[리전].amazonaws.com/1234abcd_photo.jpg"
}
```

4-1. [MQTT] 작물 질병 실시간 경고 (푸시 알림)

- 기능: 로봇이 업로드한 작물 사진의 health_status가 disease일 경우, 서버가 앱 화면에 즉각적인 경고 팝업을 띄우기 위해 발송합니다.
- [서버 → 앱] 수신 토픽: ddalgi/alert/disease/{user_id} (예: ddalgi/alert/disease/user123)
- [앱 수신] JSON Payload:
- JSON

```
{
  "log_id": 105,
  "user_id": "user123",
  "robot_id": "R_001",
  "zone": "zone01",
  "crop": "설향딸기",
  "growth_status": "RIPE",
  "health_status": "disease",
  "message": "zone01 구역에서 문제가 발견되었습니다!",
  "image_url": "https://[버킷명].s3.[리전].amazonaws.com/1234abcd_photo.jpg"
}
```

(image_url과 message를 추출하여 경고 다이얼로그나 푸시 알림 UI에 게시추천)

